

Chapter 1

Bài tập Ex2, Mid-term-2023-1-ans.pdf

Đề bài:

Cho bộ xử lý 2 GHz thực hiện tập lệnh gồm 5 lớp lệnh A, B, C, D, E với CPI lần lượt là 2, 3, 4, 6 và 10. Máy tính đang thực hiện chương trình mà các lớp lệnh A, B, C, D, E chiếm tương ứng 30%, 30%, 15%, 15% và 10% tổng số lệnh. Nếu thời gian thực hiện chương trình này là 20 giây, hãy tìm tổng số lệnh.

Lời giải:

CPI trung bình của CPU được tính như sau:

$$\text{CPI} = 0.3 * 2 + 0.3 * 3 + 0.15 * 4 + 0.15 * 6 + 0.1 * 10 = 4.0$$

Tổng số lệnh (IC) được tính như sau:

$$\begin{aligned} \text{IC} &= (\text{Thời gian thực hiện} * \text{Tốc độ xung nhịp}) / \text{CPI} \\ &= (20 \text{ giây} * 2 \text{ GHz}) / 4.0 \\ &= 10 \text{ tỷ lệnh} \end{aligned}$$

Vậy tổng số lệnh là 10 tỷ lệnh.

Câu hỏi #8cf496, IT3030---Midterm_with answers.pdf

Đề bài:

Xét hai bộ xử lý sau thực hiện cùng một công việc. P1 có tốc độ xung nhịp là 1.5 GHz, CPI trung bình là 1.8 và yêu cầu thực hiện $3.0E9$ lệnh cho công việc. P2 có tốc độ xung nhịp là 2 GHz, CPI trung bình là 1.2 và yêu cầu thực hiện $4.0E9$ lệnh cho công việc đó. P2 nhanh hơn bao nhiêu lần so với P1 khi thực hiện công việc này?

Lời giải:

Thời gian thực hiện công việc của P1:

$$\begin{aligned}\text{Thời gian P1} &= (\text{Số lượng lệnh} * \text{CPI}) / \text{Tốc độ xung nhịp} \\ &= (3.0\text{E9} * 1.8) / 1.5 \text{ GHz} \\ &= 3.6 \text{ giây}\end{aligned}$$

Thời gian thực hiện công việc của P2:

$$\begin{aligned}\text{Thời gian P2} &= (\text{Số lượng lệnh} * \text{CPI}) / \text{Tốc độ xung nhịp} \\ &= (4.0\text{E9} * 1.2) / 2 \text{ GHz} \\ &= 2.4 \text{ giây}\end{aligned}$$

Tỷ lệ giữa thời gian thực hiện của P1 và P2:

$$\begin{aligned}\text{Tỷ lệ} &= \text{Thời gian P1} / \text{Thời gian P2} \\ &= 3.6 \text{ giây} / 2.4 \text{ giây} \\ &= 1.5\end{aligned}$$

Vậy P2 nhanh hơn P1 1.5 lần.

Câu hỏi #a6b720, IT3030---Midterm_with answers.pdf

Đề bài:

Một bộ xử lý chạy chương trình với tốc độ xung nhịp là 4 GHz. Chương trình bao gồm ba loại lệnh với các đặc điểm sau:

- Loại A: 40% tổng số lệnh với CPI là 1
- Loại B: 35% tổng số lệnh với CPI là 3
- Loại C: 25% tổng số lệnh với CPI là 5

Tổng số lệnh của chương trình là 2 tỷ. Thời gian thực hiện tổng cộng là bao nhiêu?

Lời giải:

CPI trung bình của CPU:

$$\text{CPI} = 0.4 * 1 + 0.35 * 3 + 0.25 * 5 = 2.7$$

Thời gian thực hiện chương trình:

$$\begin{aligned}
 \text{Thời gian} &= (\text{Số lượng lệnh} * \text{CPI}) / \text{Tốc độ xung nhịp} \\
 &= (2 \text{ tỷ} * 2.7) / 4 \text{ GHz} \\
 &= 1.35 \text{ giây}
 \end{aligned}$$

Bài tập 1.1, Baitap.pdf

Đề bài:

Có 3 bộ xử lý P1, P2 và P3 có cùng một tập lệnh. P1, P2, P3 có tốc độ xung nhịp tương ứng là 3GHz, 2.5GHz, 4GHz và có CPI lần lượt là 1.5, 1.0, 2.2.

- Tính số triệu lệnh thực hiện trên 1 giây của từng bộ xử lý.¹
- Nếu mỗi bộ xử lý thực hiện chương trình mất 10s, tìm số chu kỳ và số lệnh tương ứng.
- Nếu chúng ta muốn giảm 30% thời gian thực hiện, dẫn đến tăng CPI thêm 20%. Khi đó tốc độ xung nhịp cho mỗi bộ xử lý tương ứng cần thiết là bao nhiêu?¹

Lời giải:

Số triệu lệnh thực hiện trên 1 giây (MIPS) của mỗi bộ xử lý được tính như sau:

$$\text{MIPS} = \text{Tốc độ xung nhịp} / \text{CPI}$$

- MIPS của P1: $\text{MIPS}_{P1} = 3 \text{ GHz} / 1.5 = 2000$
- MIPS của P2: $\text{MIPS}_{P2} = 2.5 \text{ GHz} / 1.0 = 2500$
- MIPS của P3: $\text{MIPS}_{P3} = 4 \text{ GHz} / 2.2 = 1818.18$

Câu b:

Số chu kỳ của mỗi bộ xử lý được tính bằng:

$$\text{Số chu kỳ} = \text{Thời gian thực hiện} * \text{Tốc độ xung nhịp}$$

- Số chu kỳ của P1: $3 \text{ GHz} * 10\text{s} = 30 \text{ tỷ chu kỳ}$
- Số chu kỳ của P2: $2.5 \text{ GHz} * 10\text{s} = 25 \text{ tỷ chu kỳ}$
- Số chu kỳ của P3: $4 \text{ GHz} * 10\text{s} = 40 \text{ tỷ chu kỳ}$

Số lệnh của mỗi bộ xử lý được tính bằng:

Số lệnh = Số chu kỳ / CPI

- Số lệnh của P1: $30 \text{ tỷ chu kỳ} / 1.5 = 20 \text{ tỷ lệnh}$
- Số lệnh của P2: $25 \text{ tỷ chu kỳ} / 1.0 = 25 \text{ tỷ lệnh}$
- Số lệnh của P3: $40 \text{ tỷ chu kỳ} / 2.2 = 18.18 \text{ tỷ lệnh}$

Câu c:

Thời gian thực hiện mới sau khi giảm 30% là: $10s * (1 - 30\%) = 7s$

CPI mới sau khi tăng 20% được tính như sau:

- CPI mới của P1: $1.5 * (1 + 20\%) = 1.8$
- CPI mới của P2: $1.0 * (1 + 20\%) = 1.2$
- CPI mới của P3: $2.2 * (1 + 20\%) = 2.64$

Tốc độ xung nhịp mới được tính bằng:

Tốc độ xung nhịp mới = (Số lượng lệnh * CPI mới) / Thời gian thực hiện mới

- Tốc độ xung nhịp mới của P1: $(20 \text{ tỷ lệnh} * 1.8) / 7s = 5.14 \text{ GHz}$
- Tốc độ xung nhịp mới của P2: $(25 \text{ tỷ lệnh} * 1.2) / 7s = 4.29 \text{ GHz}$
- Tốc độ xung nhịp mới của P3: $(18.18 \text{ tỷ lệnh} * 2.64) / 7s = 6.89 \text{ GHz}$

Bài tập 1.2, Baitap.pdf

Đề bài:

Có hai bộ xử lý có cùng tập lệnh. Tập lệnh được chia thành 4 lớp theo CPI (A, B, C, và D). P1 có tốc độ xung nhịp là 2.5 GHz và CPI tương ứng với các lớp lệnh là 1, 2, 3, 3; P2 có tốc độ xung nhịp là 3 GHz và CPI tương ứng với các lớp lệnh là 2, 2, 2, 2. Có chương trình với số lệnh là 10^6 lệnh, trong đó 10% lệnh lớp A, 20% lệnh lớp B, 50% lệnh lớp C, và 20% lệnh lớp D.

a. Bộ xử lý nào thực hiện nhanh hơn? b. CPI trung bình cho mỗi bộ xử lý? c. Tìm số chu kỳ cần thiết cho mỗi bộ xử lý?³

Lời giải:

Câu a:

Để so sánh hiệu năng, ta tính thời gian thực hiện chương trình của mỗi bộ xử lý.

Bộ xử lý P1:

- CPI trung bình: $\text{CPI_P1} = 0.1 * 1 + 0.2 * 2 + 0.5 * 3 + 0.2 * 3 = 2.4$
- Thời gian thực hiện: $\text{Thời gian_P1} = (10^6 \text{ lệnh} * 2.4) / 2.5 \text{ GHz} = 0.96 \text{ ms}$

Bộ xử lý P2:

- CPI trung bình: $\text{CPI_P2} = 2$
- Thời gian thực hiện: $\text{Thời gian_P2} = (10^6 \text{ lệnh} * 2) / 3 \text{ GHz} = 0.67 \text{ ms}$

Vì $\text{Thời gian_P2} < \text{Thời gian_P1}$ nên bộ xử lý P2 thực hiện nhanh hơn.

Câu b:

CPI trung bình đã được tính ở câu a:

- $\text{CPI_P1} = 2.4$
- $\text{CPI_P2} = 2$

Câu c:

Số chu kỳ cần thiết cho mỗi bộ xử lý được tính bằng:

Số chu kỳ = Số lượng lệnh * CPI

- Số chu kỳ của P1: $10^6 \text{ lệnh} * 2.4 = 2.4 \text{ triệu chu kỳ}$
- Số chu kỳ của P2: $10^6 \text{ lệnh} * 2 = 2 \text{ triệu chu kỳ}$

Chapter 3

Câu hỏi #f96f43, IT3030---Midterm_with answers.pdf

Đề bài: Chọn câu trả lời để thực hiện câu lệnh C sau trong RISC-V: `*a = b + c;`

Giả sử a là một con trỏ trong thanh ghi a1 và b, c lần lượt nằm trong thanh ghi a2, a3.

Lời giải: Để thực hiện câu lệnh C `*a = b + c;` trong hợp ngữ RISC-V, ta cần thực hiện các bước sau:

1. Tính toán `b + c`: Sử dụng lệnh `add` để cộng giá trị của thanh ghi a2 (chứa giá trị của b) và a3 (chứa giá trị của c), lưu kết quả vào một thanh ghi tạm thời (ví dụ: t0).
2. Lưu kết quả vào địa chỉ được trỏ bởi a: Sử dụng lệnh `sw` để lưu giá trị từ thanh ghi tạm thời t0 vào địa chỉ bộ nhớ được trỏ bởi thanh ghi a1 (chứa địa chỉ của a).

Vậy đáp án đúng là:

```
add t0, a2, a3
sw t0, 0(a1)
```

Câu hỏi #5e7591, IT3030---Midterm_with answers.pdf

Đề bài: Giá trị thập phân của thanh ghi s4 sau khi thực hiện các lệnh dưới đây là bao nhiêu?

```
addi s1, zero, 87
addi s2, zero, 53
xor s3, s2, zero
addi s3, s3, 1
add s4, s1, s3
```

Lời giải:

Phân tích từng lệnh:

- `addi s1, zero, 87`: Gán giá trị 87 cho thanh ghi `s1`.
- `addi s2, zero, 53`: Gán giá trị 53 cho thanh ghi `s2`.
- `xor s3, s2, zero`: XOR giá trị của `s2` với 0, kết quả lưu vào `s3` (vẫn là 53).
- `addi s3, s3, 1`: Cộng 1 vào giá trị của `s3`, kết quả là 54.
- `add s4, s1, s3`: Cộng giá trị của `s1` (87) và `s3` (54), kết quả lưu vào `s4` ($87 + 54 = 141$).

Vậy giá trị thập phân của thanh ghi `s4` là 141.

Bài tập Ex3, IT3030E-CA-Chap3-ISA-Exercises.pdf

Đề bài: Biên dịch ngược mã hợp ngữ RISC-V sau đây sang mã C tương đương. Giả sử các biến `f`, `g`, `h`, `i` và `j` được cấp phát tại các thanh ghi `x5`, `x6`, `x7`, `x28` và `x29` tương ứng và địa chỉ cơ sở của mảng `A` và `B` nằm trong các thanh ghi `x10` và `x11`.

```
slli x30, x5, 2
add x30, x10, x30
slli x31, x6, 2
add x31, x11, x31
lw x5, 0(x30)
addi x12, x30, 8
lw x30, 0(x12)
add x30, x30, x5
sw x30, 0(x31)
```

Lời giải: Phân tích từng lệnh để chuyển đổi sang mã C:

- `slli x30, x5, 2`: Dịch trái giá trị trong `x5` (biến `f`) 2 bit, tương đương với phép nhân `f` với 4. Kết quả được lưu vào `x30`.
- `add x30, x10, x30`: Cộng giá trị trong `x10` (địa chỉ cơ sở của mảng `A`) với `x30` ($f * 4$), kết quả lưu vào `x30`. Lúc này `x30` chứa địa chỉ của phần tử `A[f]`.
- `slli x31, x6, 2`: Tương tự, `x31` sẽ chứa địa chỉ của phần tử `B[g]`.
- `lw x5, 0(x30)`: Nạp giá trị từ địa chỉ trong `x30` (`A[f]`) vào thanh ghi `x5` (biến `f`).

- `addi x12, x30, 8`: Cộng 8 vào `x30` (địa chỉ của `A[f]`), kết quả lưu vào `x12`.
Lúc này `x12` chứa địa chỉ của phần tử `A[f + 2]`.
- `lw x30, 0(x12)`: Nạp giá trị từ địa chỉ trong `x12` (`A[f + 2]`) vào `x30`.
- `add x30, x30, x5`: Cộng giá trị trong `x30` (`A[f + 2]`) với `x5` (`f`), kết quả lưu vào `x30`.
- `sw x30, 0(x31)`: Lưu giá trị trong `x30` vào địa chỉ trong `x31` (`B[g]`).

Kết hợp lại, ta được mã C tương đương:

```
B[g] = A[f + 2] + f;
```

Bài tập Ex4, IT3030E-CA-Chap3-ISA-Exercises.pdf

Đề bài: Tìm định dạng và mã máy của lệnh sau: `sw x5, 32(x30)`

Lời giải:

Lệnh `sw` là lệnh store word, thuộc loại S-type trong RISC-V. Định dạng của lệnh S-type như sau:

```
imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode
31..25   | 24..20 | 19..15 | 14..12  | 11..7   | 6..0
```

Trong đó:

- `imm[11:5]` và `imm[4:0]` là các bit của immediate, kết hợp thành offset 12 bit.
- `rs2` là thanh ghi nguồn chứa giá trị cần lưu trữ (`x5`).
- `rs1` là thanh ghi nguồn chứa địa chỉ cơ sở (`x30`).
- `funct3` là mã lệnh phụ, phân biệt các lệnh S-type khác nhau. Giá trị của `funct3` cho lệnh `sw` là `010`.
- `opcode` là mã lệnh chính. Giá trị của `opcode` cho lệnh S-type là `0100011`.

Áp dụng vào lệnh `sw x5, 32(x30)`:

- `rs2 = x5 = 01001`

- `rs1 = x30 = 11110`
- `imm = 32 = 0000000000100000` -> `imm[11:5] = 0000000`, `imm[4:0] = 01000`

Ghép các trường lại với nhau, ta được mã máy của lệnh `sw x5, 32(x30)`:

0000000 | 01001 | 11110 | 010 | 01000 | 0100011

Chuyển sang hệ hexa: 0x0053A223

Chuyển đổi giữa các hệ biểu diễn

Chuyển đổi câu lệnh C sang hợp ngữ RISC-V

Câu hỏi #fe0e91, IT3030---Midterm_with answers.pdf

Đề bài: Chuyển đổi câu lệnh C sau sang hợp ngữ RISC-V: `g = h + A[8]`; Giả sử biến `g` trong thanh ghi `x5`, `h` trong thanh ghi `x6`, địa chỉ cơ sở của mảng `A` trong thanh ghi `x10`.

Lời giải:

Để truy cập phần tử `A[8]`, ta cần tính địa chỉ của phần tử này. Vì mỗi phần tử trong mảng `A` là một số nguyên 32-bit (4 byte), nên địa chỉ của `A[8]` là địa chỉ cơ sở của `A + 8 * 4`. Ta có thể sử dụng các lệnh `slli` và `add` để tính địa chỉ này. Sau đó, sử dụng lệnh `lw` để load giá trị của `A[8]` vào một thanh ghi tạm thời, và cuối cùng dùng lệnh `add` để cộng giá trị này với `h` và lưu kết quả vào `g`.

```
slli x7, x0, 3 # x7 = 8 (dịch trái 3 bit tương đương nhân với 8)
add x7, x10, x7 # x7 = địa chỉ cơ sở của A + 8 * 4
lw x7, 0(x7) # x7 = A[8]
add x5, x6, x7 # g = h + A[8]
```

Chuyển đổi hợp ngữ RISC-V sang câu lệnh C

Bài tập 3.3, Bài tập chương 3.pdf

Đề bài: Chuyển đổi đoạn mã hợp ngữ RISC-V sau đây sang câu lệnh C tương đương. Giả sử các biến `f`, `g`, `h`, `i`, `j` được gán tương ứng trong các thanh ghi `x5`, `x6`, `x7`, `x28`, `x29`; `A` và `B` là hai mảng dữ liệu các phần tử số nguyên 32-bit có địa chỉ cơ sở tương ứng nằm trong các thanh ghi `x10` và `x11`.

```
slli x7, x29, 1
add x5, x7, x28
add x5, x5, x6
```

Lời giải:

Phân tích từng lệnh:

- `slli x7, x29, 1`: Dịch trái giá trị của thanh ghi `x29` (biến `j`) sang 1 bit, tương đương với phép nhân `j` với 2. Kết quả được lưu vào thanh ghi `x7`.
- `add x5, x7, x28`: Cộng giá trị của thanh ghi `x7` ($2 * j$) với giá trị của thanh ghi `x28` (biến `i`), kết quả được lưu vào thanh ghi `x5` (biến `f`).
- `add x5, x5, x6`: Cộng giá trị của thanh ghi `x5` ($2 * j + i$) với giá trị của thanh ghi `x6` (biến `g`), kết quả được lưu vào thanh ghi `x5` (biến `f`).

Ghép các lệnh lại, ta được câu lệnh C tương đương:

```
f = 2 * j + i + g;
```

Chuyển đổi lệnh RISC-V sang mã máy và ngược lại

Bài tập 3.4:

Đề bài: Chuyển đổi các lệnh RISC-V sau đây sang dạng mã máy viết theo hệ hexa:

```
lw x6, 32(x0)
add x5, x29, x30
addi x5, x28, -12
sub x5, x7, x31
```

Lời giải:

- `lw x6, 32(x0):`
 - Loại lệnh: I-type
 - opcode: 0000011
 - rd: x6 = 00110
 - funct3: 010
 - rs1: x0 = 00000
 - imm: 32 = 0000000000100000
 - Mã máy: 000000000010000000000110010
- `add x5, x29, x30:`
 - Loại lệnh: R-type
 - opcode: 0110011
 - rd: x5 = 00101
 - funct3: 000
 - rs1: x29 = 11101
 - rs2: x30 = 11110
 - funct7: 0000000
 - Mã máy: 00000001111011101000001010110011
- `addi x5, x28, -12:`
 - Loại lệnh: I-type
 - opcode: 0010011
 - rd: x5 = 00101
 - funct3: 000
 - rs1: x28 = 11100
 - imm: -12 = 111111111110100
 - Mã máy: 11111111111010011100001010010011

- `sub x5, x7, x31:`

- Loại lệnh: R-type
- opcode: 0110011
- rd: x5 = 00101
- funct3: 000
- rs1: x7 = 00111
- rs2: x31 = 11111
- funct7: 0100000
- Mã máy: 01000001111100111000001010110011

Bài tập 3.5:

Đề bài: Chuyển đổi các lệnh mã máy RISC-V sau đây sang dạng lệnh hợp ngữ:

0x2237FFF1
0x02F34022
0x8E93FFE8

Lời giải:

- 0x2237FFF1:
 - Phân tích mã máy: 0010 0011 1111 1111 1111 1111 1111 0001
 - opcode: 0010011 -> I-type, addi
 - rd: 00001 -> x1
 - funct3: 000
 - rs1: 00111 -> x7
 - imm: 1111 1111 1111 1111 1111 -> -1
 - Lệnh hợp ngữ: `addi x1, x7, -1`
- 0x02F34022:
 - Phân tích mã máy: 0000 0000 1111 0011 0100 0000 00110 0100011

- opcode: 0100011 -> S-type, sw
 - imm[11:5]: 0000000
 - rs2: 01111 -> x15
 - rs1: 00110 -> x6
 - funct3: 010
 - imm[4:0]: 01000
 - imm: 000000001000 = 8
 - Lệnh hợp ngữ: `sw x15, 8(x6)`
- 0x8E93FFE8:
 - Phân tích mã máy: 1000 1110 1001 0011 1111 1111 1111 1000
 - opcode: 1100111 -> B-type, bge
 - imm[12]: 1
 - imm[10:5]: 000111
 - rs2: 00110 -> x6
 - rs1: 10010 -> x18
 - funct3: 100
 - imm[4:1]: 1111
 - imm[11]: 1
 - imm: 1000 1111 1111 1000 = -104 (tính toán giá trị của imm từ biểu diễn bù 2)
 - Lệnh hợp ngữ: `bge x18, x6, -104`

Phân tích và thực thi mã:

Xác định giá trị thanh ghi đích sau khi thực hiện một đoạn mã hợp ngữ RISC-V

Câu hỏi #5e7591, IT3030---Midterm_with answers.pdf

Đề bài: Giá trị thập phân của thanh ghi `x5` sau khi thực hiện các lệnh dưới đây là bao nhiêu?

```
addi x6, zero, 87
addi x7, zero, 53
xor x8, x7, zero
addi x8, x8, 1
add x5, x6, x8
```

Lời giải:

Phân tích từng lệnh:

- `addi x6, zero, 87`: Gán giá trị 87 cho thanh ghi `x6`.
- `addi x7, zero, 53`: Gán giá trị 53 cho thanh ghi `x7`.
- `xor x8, x7, zero`: XOR giá trị của `x7` với 0, kết quả lưu vào `x8` (vẫn là 53).
- `addi x8, x8, 1`: Cộng 1 vào giá trị của `x8`, kết quả là 54.
- `add x5, x6, x8`: Cộng giá trị của `x6` (87) và `x8` (54), kết quả lưu vào `x5` ($87 + 54 = 141$).

Vậy giá trị thập phân của thanh ghi `x5` là 141.

Câu hỏi #1b55cf, IT3030---Midterm_with answers.pdf

Đề bài: Cho đoạn mã hợp ngữ RISC-V dưới đây. Giá trị của thanh ghi `x7` sau khi thực hiện đoạn mã là bao nhiêu (dưới dạng hexa)?

```
li x5, 1
lui x6, 0xFFFFF
ori x6, x6, 2047
add x7, x5, x6
```

Lời giải:

Phân tích từng lệnh:

- `li x5, 1`: Gán giá trị 1 cho thanh ghi `x5`.

- `lui x6, 0xFFFFF`: Load giá trị 0xFFFFF vào 20 bit cao của thanh ghi `x6`, 12 bit thấp bằng 0. Lúc này `x6` có giá trị là `0xFFFFF000`.
- `ori x6, x6, 2047`: OR giá trị của `x6` với 2047, kết quả lưu vào `x6`. 2047 trong hệ hexa là `0x7FF`, do đó `x6` sẽ có giá trị là `0xFFFFF7FF`.
- `add x7, x5, x6`: Cộng giá trị của `x5` (1) và `x6` (`0xFFFFF7FF`), kết quả lưu vào `x7`.

Kết quả phép cộng là `0xFFFFF800`.

Vậy giá trị của thanh ghi `x7` là `0xFFFFF800`.

Phân tích đoạn chương trình RISC-V để tính số lệnh được thực hiện và giá trị thanh ghi

Bài tập 3.7, Bài tập chương 3.pdf

Đề bài: Cho đoạn chương trình vòng lặp viết bằng hợp ngữ RISC-V sau đây:

```
addi x6, zero, 8
add x7, zero, zero
LOOP:
slt x8, zero, x6
beq x8, zero, DONE
addi x7, x7, 3
slli x7, x7, 1
addi x6, x6, -1
j LOOP
DONE:
```

a. Tính số lệnh được thực hiện khi chạy đoạn chương trình trên. b. Xác định giá trị thanh ghi `x7` sau khi thực hiện đoạn chương trình trên.

Lời giải:

Câu a:

- Vòng lặp sẽ thực hiện 8 lần, mỗi lần lặp thực hiện 6 lệnh (`slt`, `beq`, `addi`, `slli`, `addi`, `j`).

- Sau khi thoát vòng lặp, thực hiện thêm 1 lệnh (add).

Tổng số lệnh được thực hiện là: $8 \text{ lần} * 6 \text{ lệnh/lần} + 1 \text{ lệnh} = 49 \text{ lệnh}$

Câu b:

Phân tích vòng lặp:

- `x6` được khởi tạo bằng 8, đóng vai trò biến đếm vòng lặp.
- `x7` được khởi tạo bằng 0, là biến lưu trữ kết quả.
- Mỗi lần lặp, `x7` được cộng thêm 3, sau đó dịch trái 1 bit (nhân đôi).

Giá trị của `x7` sau mỗi lần lặp:

Lần lặp	x7 trước khi cộng	x7 sau khi cộng	x7 sau khi dịch trái
1	0	3	6
2	6	9	18
3	18	21	42
4	42	45	90
5	90	93	186
6	186	189	378
7	378	381	762
8	762	765	1530

Vậy giá trị của thanh ghi `x7` sau khi thực hiện đoạn chương trình là 1530.

Bài tập 3.8, Bài tập chương 3.pdf

Đề bài: Cho đoạn chương trình vòng lặp viết bằng hợp ngữ RISC-V sau đây:

```
LOOP:
    slt x8, zero, x6
```



```

bne x8, zero, ELSE
j DONE
ELSE:
addi x7, x7, 2
addi x6, x6, -1
j LOOP
DONE:

```

a. Giả thiết các thanh ghi `x6`, `x7` được khởi tạo các giá trị ban đầu là `x6 = 18`, `x7 = 0`, hãy xác định giá trị thanh ghi `x7` sau khi thực hiện đoạn chương trình trên. b. Với vòng lặp hợp ngữ trên, giả sử thanh ghi `x6` được khởi tạo giá trị bằng N (với N nguyên dương), hãy xác định khi thực hiện đoạn chương trình trên thì có bao nhiêu lệnh được thực hiện?

Lời giải:

Câu a:

- Vòng lặp sẽ thực hiện 18 lần.
- Mỗi lần lặp, nếu `x6 > 0`, `x7` được cộng thêm 2.

Do đó, `x7` sẽ được cộng thêm 2 tổng cộng 18 lần.

Vậy giá trị của thanh ghi `x7` sau khi thực hiện đoạn chương trình là $2 * 18 = 36$.

Câu b:

- Mỗi lần lặp thực hiện 4 lệnh (`slt`, `bne`, `addi`, `j`).
- Khi `x6` bằng 0, thực hiện 2 lệnh (`slt`, `j`).

Tổng số lệnh được thực hiện là: $N \text{ lần} * 4 \text{ lệnh/lần} + 2 \text{ lệnh} = 4N + 2$

Cài đặt các cấu trúc điều khiển:

Cài đặt câu lệnh điều kiện if-then-else trong hợp ngữ RISC-V

Bài tập Ex2, IT3030E-CA-Chap3-ISA-Exercises.pdf

Đề bài: Viết chương trình hợp ngữ RISC-V để thực hiện câu lệnh C sau:

```
if (i <= j) {
    x = x + 1;
    z = 1;
} else {
    y = y - 1;
    z = 2 * z;
}
```

Giả sử các biến `i`, `j`, `x`, `y`, `z` được lưu lần lượt trong các thanh ghi `x5`, `x6`, `x7`, `x8`, `x9`.

Lời giải:

```
        blt x6, x5, else    # nếu j < i, nhảy đến else
        addi x7, x7, 1      # x = x + 1
        addi x9, x0, 1      # z = 1
        j endif            # nhảy đến endif
else:
        addi x8, x8, -1     # y = y - 1
        slli x9, x9, 1      # z = 2 * z
endif:
        # ...
```

Cài đặt vòng lặp trong hợp ngữ RISC-V

Bài tập Ex2, IT3030E-CA-Chap3-ISA-Exercises.pdf

Đề bài: Viết chương trình hợp ngữ RISC-V để tính tổng các phần tử của mảng `A`.

```
loop:
    i = i + step;
    sum = sum + A[i];
    if (i != n) goto loop;
```

Giả sử các biến `i`, `step`, `sum`, `n` và địa chỉ cơ sở của mảng `A` được lưu lần lượt trong các thanh ghi `x5`, `x6`, `x7`, `x8`, `x9`.

Lời giải:

```
loop:
    add x5, x5, x6          # i = i + step
    slli x10, x5, 2         # x10 = i * 4
    add x10, x10, x9        # x10 = địa chỉ của A[i]
```

```

lw x10, 0(x10)      # x10 = A[i]
add x7, x7, x10      # sum = sum + A[i]
bne x5, x8, loop     # nếu i != n, quay lại loop

```

Cài đặt câu lệnh switch/case trong hợp ngữ RISC-V

Bài tập Ex3, IT3030E-CA-Chap3-ISA-Exercises.pdf

Đề bài: Viết chương trình hợp ngữ RISC-V để thực hiện câu lệnh C sau:

```

switch (test) {
  case 0:
    a = a + 1;
    break;
  case 1:
    a = a - 1;
    break;
  case 2:
    b = 2 * b;
    break;
}

```

Giả sử các biến `test`, `a`, `b` được lưu lần lượt trong các thanh ghi `x5`, `x6`, `x7`.

Lời giải:

```

addi x8, x0, 0      # x8 = 0
addi x9, x0, 1      # x9 = 1
addi x10, x0, 2     # x10 = 2
beq x5, x8, case0   # nếu test == 0, nhảy đến case0
beq x5, x9, case1   # nếu test == 1, nhảy đến case1
beq x5, x10, case2  # nếu test == 2, nhảy đến case2
j default           # nhảy đến default

case0:
  addi x6, x6, 1     # a = a + 1
  j continue        # nhảy đến continue
case1:
  addi x6, x6, -1    # a = a - 1
  j continue        # nhảy đến continue
case2:
  slli x7, x7, 1     # b = 2 * b
  j continue        # nhảy đến continue
default:
continue:
  # ...

```

Chapter 4

Câu hỏi #94ec4c, IT3030---Midterm_with answers.pdf

Đề bài: Biểu diễn số thập phân -31.75 dưới dạng số dấu phẩy động IEEE 754 đơn độ chính xác.

Lời giải:

1. Chuyển đổi phần nguyên sang hệ nhị phân: $31 = 11111_2$
2. Chuyển đổi phần thập phân sang hệ nhị phân: $0.75 = 0.11_2$
3. Kết hợp phần nguyên và phần thập phân: 11111.11_2
4. Chuẩn hóa số nhị phân: 1.111111×2^4
5. Xác định các trường của số dấu phẩy động:
 - Dấu: 1 (vì số âm)
 - Phần định trị: 111111 (lấy phần sau dấu chấm của số đã chuẩn hóa)
 - Số mũ: $4 + 127 = 131 = 10000011_2$ (127 là độ lệch bias cho số mũ)
6. Ghép các trường lại với nhau:

Dấu	Số mũ	Phần định trị
1	10000011	111111000000000000000000

Kết quả: Biểu diễn IEEE 754 đơn độ chính xác của -31.75 là

11000001111111100000000000000000.

Câu hỏi #386374, IT3030---Midterm_with answers.pdf

Đề bài: Biểu diễn số thập phân 12.25 dưới dạng số dấu phẩy động IEEE 754 đơn độ chính xác.

Lời giải:

1. Chuyển đổi phần nguyên sang hệ nhị phân: $12 = 1100_2$
2. Chuyển đổi phần thập phân sang hệ nhị phân: $0.25 = 0.01_2$
3. Kết hợp phần nguyên và phần thập phân: 1100.01_2
4. Chuẩn hóa số nhị phân: 1.10001×2^3
5. Xác định các trường của số dấu phẩy động:
 - Dấu: 0 (vì số dương)
 - Phần định trị: 10001 (lấy phần sau dấu chấm của số đã chuẩn hóa)
 - Số mũ: $3 + 127 = 130 = 10000010_2$ (127 là độ lệch bias cho số mũ)
6. Ghép các trường lại với nhau:

Dấu	Số mũ	Phần định trị
0	10000010	1000100000000000000000

Kết quả: Biểu diễn IEEE 754 đơn độ chính xác của 12.25 là

010000010100010000000000000000.

Câu hỏi #bd4c95, IT3030---Midterm_with answers.pdf

Đề bài: Cho số dấu phẩy động đơn độ chính xác

010000010101000000000000000000. Xác định giá trị thập phân của số này.

Lời giải:

1. Tách các trường:

- Dấu: 0 (dương)
- Số mũ: $10000010_2 = 130$
- Phần định trị: 1010000000000000000000

2. Tính số mũ: $130 - 127 = 3$

3. Tạo lại số nhị phân đã chuẩn hóa: 1.101×2^3

4. Chuyển đổi sang hệ thập phân: $1101_2 = 13$

Kết quả: Giá trị thập phân của số dấu phẩy động là 13.

Câu hỏi #f2d536, IT3030---Midterm_with answers.pdf

Đề bài: Tìm kết quả của phép toán $10110110 + 01101101$ với hai toán hạng là các số nguyên có dấu 8 bit. Chọn một câu trả lời.

Lời giải:

1. Xác định số âm: Số đầu tiên (10110110) là số âm vì bit dấu (bit ngoài cùng bên trái) là 1.
2. Chuyển đổi sang dạng bù 2:
 - Đảo tất cả các bit của số âm: 01001001

- Cộng 1 vào kết quả: 01001010

3. Thực hiện phép cộng:

```

01001010
+ 01101101
-----
10110111

```

4. Xác định số âm: Kết quả là số âm vì bit dấu là 1.

Chuyển đổi sang dạng bù 2:

- Đảo tất cả các bit: 01001000
- Cộng 1 vào kết quả: 01001001

5. Chuyển đổi sang hệ thập phân: $01001001_2 = 73$

Kết quả: Kết quả của phép toán là -73.

Câu hỏi #e83dac, IT3030---Midterm_with answers.pdf

Đề bài: Xác định trường hợp tràn số trong phép cộng số nguyên có dấu 32 bit.

Lời giải:

Tràn số xảy ra khi kết quả của phép cộng hai số nguyên nằm ngoài phạm vi biểu diễn của số nguyên có dấu 32 bit.

- Cộng hai số dương: Tràn số xảy ra khi kết quả là số âm (bit dấu là 1).
- Cộng hai số âm: Tràn số xảy ra khi kết quả là số dương (bit dấu là 0).

Kết quả: Trường hợp tràn số là cộng hai số dương cho kết quả là số âm, hoặc cộng hai số âm cho kết quả là số dương.

Bài tập 1: Biểu diễn số nguyên có dấu và chuyển đổi sang hệ hexa

1. Biểu diễn số nguyên có dấu theo mã bù hai:

- Bước 1: Biểu diễn số nguyên dưới dạng nhị phân.
- Bước 2: Nếu số nguyên là số âm, đảo ngược các bit trong biểu diễn nhị phân và cộng thêm 1.
- Bước 3: Thêm các bit 0 vào đầu để có đủ số bit theo yêu cầu (8-bit hoặc 16-bit).

Ví dụ: Biểu diễn số -43 theo mã bù hai 8-bit:

- Nhị phân của 43: 101011
- Đảo ngược các bit: 010100
- Cộng thêm 1: 010101
- Thêm các bit 0 vào đầu: 11010101

2. Chuyển đổi sang hệ hexa:

- Bước 1: Chia biểu diễn nhị phân thành các nhóm 4 bit.
- Bước 2: Chuyển đổi mỗi nhóm 4 bit sang giá trị hexa tương ứng.

Ví dụ: Chuyển đổi 11010101 sang hệ hexa:

- Chia thành nhóm: 1101 0101
- Chuyển đổi sang hexa: D5

Kết quả:

Số nguyên	Mã bù hai 8-bit	Hexa 8-bit	Mã bù hai 16-bit	Hexa 16-bit
+104	01101000	68	000000001101000	0068
-43	11010101	D5	1111111111010101	FFD5
+1041	0000010000010001	0411	0000010000010001	0411
-528	1111111000000000	FE00	1111111111111100000000	FFFE00

-1	11111111	FF	1111111111111111	FFFF
----	----------	----	------------------	------

Bài tập 2: Phép cộng số nguyên có dấu 8 bit

Cho đoạn chương trình:

```
i = -93;
```

```
j = -78;
```

```
k = i + j;
```

a) Biểu diễn i và j dưới dạng nhị phân theo mã bù hai:

- $i = -93 = 10100011$
- $j = -78 = 10110010$

b) Tính k theo nhị phân và kết quả dưới dạng thập phân:

- $k = i + j = 10100011 + 10110010 = 101010101$
- Do k là số nguyên có dấu 8-bit, ta bỏ bit tràn, kết quả là $01010101 = 85$
- Chuyển đổi 01010101 sang dạng bù hai: $10101011 = -83$

Giải thích: Kết quả nhận được là -83 do hiện tượng tràn số. Tổng của hai số âm -93 và -78 nằm ngoài phạm vi biểu diễn của số nguyên có dấu 8-bit.

Bài tập 3: Nhân hai số nguyên không dấu 8 bit

$M \times Q = 25 \times 18 = 18 \times 25$

Thực hiện phép nhân theo thuật giải nhân số nguyên không dấu 8-bit:

```
00011001 (25)
```

```
x 00010010 (18)
```

```
-----
```

```

00000000
00011001
00000000
00000000
00011001
00000000
00000000
00000000
-----
000010110110 (450)

```

Kết quả: $25 \times 18 = 450$

Bài tập 4: Biểu diễn số thực sang số dấu phẩy động IEEE 754

Biểu diễn các số thực sau đây về dạng số dấu phẩy động IEEE754-2008 32-bit viết theo dạng số Hexa:

- $X = 450$
- $Y = -46.5$
- $Z = 1/32$
- $V = 0.2$

Cách thực hiện:

1. Xác định dấu: 0 cho số dương, 1 cho số âm.
2. Biểu diễn phần định trị dưới dạng nhị phân.
3. Chuẩn hóa phần định trị: Dịch dấu chấm động về phía sau chữ số 1 đầu tiên.
4. Tính phần mũ: Lấy số vị trí dịch chuyển cộng với 127 (đối với chuẩn 32-bit).

5. Biểu diễn phần mũ dưới dạng nhị phân.
6. Ghép các phần lại với nhau: 1 bit dấu + 8 bit mũ + 23 bit phần định trị.
7. Chuyển đổi sang hệ hexa.

Kết quả:

Số thực	Dấu	Phần định trị	Mũ	Số dấu phẩy động (nhị phân)	Hexa
$X = 450$	0	111000010	$8 + 127 = 135 = 10000111$	0100001111110000100 000000000000000	43F01000
$Y = -46.5$	1	101110.1	$5 + 127 = 132 = 10000100$	110000100101110100 000000000000000	C25D0000
$Z = 1/32$	0	1	$-5 + 127 = 122 = 01111010$	001111010100000000 000000000000000	3D400000
$V = 0.2$	0	110011001100 11001100110	$-3 + 127 = 124 = 01111100$	0011111001100110011 0011001100110	3E666666

Bài tập 5: Xác định giá trị thập phân của số dấu phẩy động IEEE 754

Cho các số dấu phẩy động theo chuẩn IEEE754 32-bit được viết theo dạng số Hexa:

- $M = 0xC1E00000$
- $N = 0x3F500000$
- $P = 0x40000000$

Cách thực hiện:

1. Chuyển đổi số hexa sang nhị phân.
2. Tách các phần: 1 bit dấu + 8 bit mũ + 23 bit phần định trị.

3. Xác định dấu: 0 là dương, 1 là âm.
4. Tính phần mũ: Chuyển đổi phần mũ sang dạng thập phân và trừ đi 127.
5. Tính phần định trị: Chuyển đổi phần định trị sang dạng thập phân, thêm 1 vào đầu và dịch dấu chấm động sang phải số vị trí bằng phần mũ.

Kết quả:

Số hexa	Nhị phân	D ấ u	Mũ	Phần định trị	Giá trị thập phân
M = 0xC1E00000	110000011111000000 0000000000000000	1	10000011 = 131 - 127 = 4	1110000 0...	-14.0
N = 0x3F500000	00111111010100000 0000000000000000	0	01111110 = 126 - 127 = -1	1010000 0...	0.625
P = 0x40000000	01000000000000000 0000000000000000	0	10000000 = 128 - 127 = 1	1000000 0...	2.0

Chapter 5

Bài tập Ex1: Xác định giá trị của các tín hiệu điều khiển trong CPU đơn chu kỳ

Yêu cầu: Xác định giá trị của các tín hiệu điều khiển trong CPU đơn chu kỳ cho lệnh AND.

Phân tích:

Lệnh AND là lệnh thuộc loại R-type, có format như sau:

```
AND Rd, Rs1, Rs2
```

Lệnh này thực hiện phép AND bitwise giữa giá trị trong thanh ghi Rs1 và Rs2, sau đó lưu kết quả vào thanh ghi Rd.

Xác định giá trị các tín hiệu điều khiển:

Dựa vào sơ đồ datapath của CPU đơn chu kỳ và format của lệnh AND, ta có thể xác định giá trị của các tín hiệu điều khiển như sau:

Tín hiệu điều khiển	Giá trị	Giải thích
RegWrite	1	Cho phép ghi kết quả vào thanh ghi đích Rd
ALUSrc	0	Chọn đầu vào thứ hai của ALU từ thanh ghi Rs2
MemRead	0	Không đọc dữ liệu từ bộ nhớ
MemWrite	0	Không ghi dữ liệu vào bộ nhớ
MemtoReg	0	Chọn đầu ra của ALU để ghi vào thanh ghi đích
Branch	0	Không phải lệnh rẽ nhánh
ALUOp	AND	ALU thực hiện phép AND bitwise

Các khối/thành phần tạo ra output hữu ích cho lệnh AND:

- Instruction memory: Cung cấp lệnh AND cho CPU.
- Registers: Cung cấp giá trị của các thanh ghi Rs1 và Rs2.
- ALU: Thực hiện phép AND bitwise.
- Mux: Chọn đầu ra của ALU để ghi vào thanh ghi đích.
- Control: Tạo ra các tín hiệu điều khiển.

Bài tập Ex2: Phân tích ảnh hưởng của việc cắt các đường tín hiệu trong CPU

Yêu cầu: Phân tích ảnh hưởng của việc cắt các đường tín hiệu (1), (2), (3) trong CPU.

Phân tích:

- Đường tín hiệu (1): Đường này kết nối đầu ra của ALU với đầu vào của Data memory. Việc cắt đường này sẽ ngăn cản việc ghi dữ liệu vào bộ nhớ.
 - Lệnh bị lỗi: `sw` (store word) - Lệnh này ghi dữ liệu vào bộ nhớ.
 - Lệnh vẫn hoạt động: `add` - Lệnh này chỉ thực hiện phép cộng và ghi kết quả vào thanh ghi.
- Đường tín hiệu (2): Đường này kết nối đầu ra của Imm Gen với đầu vào thứ hai của ALU. Việc cắt đường này sẽ ngăn cản việc sử dụng giá trị immediate trong các lệnh.
 - Lệnh bị lỗi: `addi` - Lệnh này cộng giá trị trong thanh ghi với một giá trị immediate.
 - Lệnh vẫn hoạt động: `add` - Lệnh này cộng giá trị của hai thanh ghi.
- Đường tín hiệu (3): Đường này kết nối đầu ra của PC với đầu vào của Adder. Việc cắt đường này sẽ ngăn cản việc cập nhật địa chỉ PC.
 - Lệnh bị lỗi: Tất cả các lệnh - CPU sẽ không thể nạp lệnh tiếp theo để thực thi.

- Lệnh vẫn hoạt động: Không có lệnh nào có thể hoạt động.

Bài tập Ex3: Tính toán thời gian chu kỳ và so sánh hiệu năng CPU đơn chu kỳ khi bổ sung thêm khối nhân phần cứng

Yêu cầu: Tính toán thời gian chu kỳ và so sánh hiệu năng CPU đơn chu kỳ khi bổ sung thêm khối nhân phần cứng.

Phân tích:

Thời gian chu kỳ của CPU đơn chu kỳ được xác định bởi độ trễ của đường dẫn tới hạn (critical path), tức là đường dẫn có độ trễ lớn nhất. Khi chưa có khối nhân phần cứng, đường dẫn tới hạn bao gồm các khối sau:

`I-Mem -> Regs (read) -> ALU -> Regs (write)`

Độ trễ của đường dẫn này là:

$$400\text{ps} + 350\text{ps} + 120\text{ps} + 350\text{ps} = 1220\text{ps}$$

Khi bổ sung khối nhân phần cứng, độ trễ của ALU tăng thêm 300ps, do đó độ trễ của đường dẫn tới hạn mới là:

$$400\text{ps} + 350\text{ps} + (120\text{ps} + 300\text{ps}) + 350\text{ps} = 1520\text{ps}$$

Kết luận:

- Thời gian chu kỳ khi chưa có khối nhân phần cứng: 1220ps
- Thời gian chu kỳ khi có khối nhân phần cứng: 1520ps

Việc bổ sung khối nhân phần cứng làm tăng thời gian chu kỳ, do đó làm giảm tốc độ CPU. Tuy nhiên, khối nhân phần cứng giúp giảm số lượng lệnh cần thực thi, do đó có thể cải thiện hiệu năng tổng thể của CPU.

Đánh giá:

Việc bổ sung khối nhân phần cứng có thể là một lựa chọn thiết kế tốt nếu lợi ích từ việc giảm số lượng lệnh cần thực thi lớn hơn chi phí tăng thời gian chu kỳ.

Bài tập Ex4: Tính toán tỷ lệ sử dụng bộ nhớ trong CPU đa chu kỳ

Yêu cầu: Tính toán tỷ lệ sử dụng bộ nhớ dữ liệu trong CPU đa chu kỳ.

Phân tích:

CPU đa chu kỳ chỉ sử dụng bộ nhớ dữ liệu trong các lệnh load (`lw`) và store (`sw`).

Theo bảng thống kê, tỷ lệ các lệnh load và store là:

`lw`: 25%
`sw`: 10%

Tổng tỷ lệ sử dụng bộ nhớ dữ liệu là:

$$25\% + 10\% = 35\%$$

Kết luận:

Tỷ lệ sử dụng bộ nhớ dữ liệu trong CPU đa chu kỳ là 35%.

Bài tập Ex5: Tính toán thời gian chu kỳ, độ trễ, tỷ lệ sử dụng bộ nhớ và cổng ghi thanh ghi trong CPU đa chu kỳ

Yêu cầu: Tính toán thời gian chu kỳ, độ trễ, tỷ lệ sử dụng bộ nhớ và cổng ghi thanh ghi trong CPU đa chu kỳ.

Phân tích:

(a) Thời gian chu kỳ:

- CPU đa chu kỳ: Thời gian chu kỳ được xác định bởi độ trễ của giai đoạn pipeline dài nhất. Trong trường hợp này, giai đoạn ID có độ trễ dài nhất là 350ps.

- CPU đơn chu kỳ: Thời gian chu kỳ được xác định bởi độ trễ của toàn bộ datapath. Trong trường hợp này, độ trễ của datapath là $250\text{ps} + 350\text{ps} + 150\text{ps} + 300\text{ps} + 200\text{ps} = 1250\text{ps}$.

(b) Độ trễ của lệnh LW:

- CPU đa chu kỳ: Lệnh LW phải đi qua tất cả các giai đoạn pipeline, do đó độ trễ là $250\text{ps} + 350\text{ps} + 150\text{ps} + 300\text{ps} + 200\text{ps} = 1250\text{ps}$.
- CPU đơn chu kỳ: Lệnh LW được thực thi trong một chu kỳ duy nhất, do đó độ trễ cũng là 1250ps .

(c) Tỷ lệ sử dụng bộ nhớ lệnh:

- CPU đa chu kỳ: Trong mỗi chu kỳ, CPU đều nạp một lệnh mới từ bộ nhớ lệnh, do đó tỷ lệ sử dụng là 100%.
- CPU đơn chu kỳ: CPU chỉ nạp lệnh mới khi bắt đầu thực thi một lệnh, do đó tỷ lệ sử dụng thấp hơn CPU đa chu kỳ.

(d) Tỷ lệ sử dụng bộ nhớ dữ liệu và cổng ghi thanh ghi:

- Bộ nhớ dữ liệu: CPU chỉ sử dụng bộ nhớ dữ liệu trong các lệnh load (`lw`) và store (`sw`). Theo bảng thống kê, tỷ lệ các lệnh load và store là $20\% + 15\% = 35\%$.
- Cổng ghi thanh ghi: CPU chỉ ghi vào thanh ghi trong các lệnh ALU (`alu`) và load (`lw`). Theo bảng thống kê, tỷ lệ các lệnh này là $45\% + 20\% = 65\%$.

Kết luận:

Đặc trưng	CPU đa chu kỳ	CPU đơn chu kỳ
Thời gian chu kỳ	350ps	1250ps
Độ trễ lệnh LW	1250ps	1250ps
Tỷ lệ sử dụng bộ nhớ lệnh	100%	Thấp hơn CPU đa chu kỳ

Tỷ lệ sử dụng bộ nhớ dữ liệu	35%	35%
Tỷ lệ sử dụng cổng ghi thanh ghi	65%	65%

Bài tập Ex6: Phát hiện và xác định các dạng xung đột (hazard) trong pipeline

Yêu cầu: Phát hiện và xác định các dạng xung đột (hazard) trong pipeline.

Phân tích:

(a) Đoạn mã:

```
lw t0, 0(t0)
add t1, t0, t0
```

- Xung đột dữ liệu (data hazard): Lệnh `add` cần giá trị của thanh ghi `t0` được load bởi lệnh `lw` ở chu kỳ trước đó. Đây là xung đột RAW (Read After Write).

(b) Đoạn mã:

```
add t1, t0, t0
addi t2, t0, 5
addi t4, t1, 4
```

- Xung đột dữ liệu (data hazard): Lệnh `addi t2, t0, 5` cần giá trị của thanh ghi `t0` được sử dụng bởi lệnh `add` ở chu kỳ trước đó. Đây là xung đột RAW (Read After Write).
- Xung đột dữ liệu (data hazard): Lệnh `addi t4, t1, 4` cần giá trị của thanh ghi `t1` được ghi bởi lệnh `add` ở chu kỳ trước đó. Đây là xung đột RAW (Read After Write).

Bài tập Ex7: So sánh hiệu năng của CPU đa chu kỳ với các phiên bản khác nhau về kỹ thuật forwarding

Yêu cầu: So sánh hiệu năng của CPU đa chu kỳ với các phiên bản khác nhau về kỹ thuật forwarding.

Phân tích:

(a) Xung đột tiềm ẩn:

Đoạn mã có các xung đột dữ liệu RAW (Read After Write) do lệnh sau phụ thuộc vào kết quả của lệnh trước.

(b) So sánh hiệu năng:

- CPU1 (không có forwarding): CPU phải đợi 2 chu kỳ để dữ liệu được ghi vào thanh ghi rồi mới đọc được cho lệnh tiếp theo.
- CPU2 (forwarding đầy đủ): CPU có thể chuyển kết quả từ giai đoạn EX hoặc MEM của lệnh trước đến giai đoạn EX của lệnh hiện tại, giảm thiểu thời gian chờ.
- CPU3 (chỉ forwarding ALU-ALU): CPU chỉ có thể chuyển kết quả từ giai đoạn EX của lệnh trước đến giai đoạn EX của lệnh hiện tại nếu cả hai lệnh đều sử dụng ALU.

Kết luận:

CPU2 (forwarding đầy đủ) là nhanh nhất do giảm thiểu được tối đa thời gian chờ do xung đột dữ liệu. Tuy nhiên, kỹ thuật forwarding phức tạp và tốn kém về mặt phần cứng. CPU3 là một sự thỏa hiệp giữa hiệu năng và chi phí. CPU1 là chậm nhất do không có forwarding.

Bài tập Ex8: Vẽ sơ đồ pipeline và tính toán thời gian chu kỳ của CPU đa chu kỳ

Yêu cầu: Vẽ sơ đồ pipeline và tính toán thời gian chu kỳ của CPU đa chu kỳ.

Phân tích:

(a) Thời gian chu kỳ:

Chu kÿ	1	2	3	4	5	6	7	8	9	10	11

`lw`		IF		ID		EX		MEM		WB									
`and`				IF		ID		EX		MEM		WB							
`lw`						IF		ID		EX		MEM		WB					
`lw`								IF		ID		EX		MEM		WB			
`beq`										IF		ID		EX		MEM		WB	
`lw`												IF		ID		EX		MEM	
...												...							

(b) Tỷ lệ sử dụng các giai đoạn pipeline:

Do không có xung đột, nên tất cả các giai đoạn pipeline đều được sử dụng trong mỗi chu kỳ.

Tỷ lệ sử dụng:

- IF: 100%
- ID: 100%
- EX: 100%
- MEM: 100%
- WB: 100%

(c) CPI trung bình:

CPI (Cycles Per Instruction) là số chu kỳ trung bình để thực thi một lệnh. Do không có xung đột, nên CPI trung bình là 1.

Chapter 6

1. Tính toán các thông số của bộ nhớ

Xác định số byte nhớ tối đa, địa chỉ đầu, địa chỉ cuối:

Ví dụ: Bài tập 6.1, bai-tap-chuong-6.pdf

Đề bài: Máy tính dùng 32 bit địa chỉ để đánh địa chỉ cho bộ nhớ theo byte; bus dữ liệu để kết nối với bộ nhớ chính là 32 bit. Hãy cho biết:

a. Số byte nhớ tối đa được đánh địa chỉ? Địa chỉ đầu và địa chỉ cuối dưới dạng hexa?¹

Lời giải:

- Số byte nhớ tối đa: Với 32 bit địa chỉ, máy tính có thể đánh địa chỉ tối đa 2^{32} byte = 4GB.
- Địa chỉ đầu: 0x00000000
- Địa chỉ cuối: 0xFFFFFFFF

Tính toán số bit tối thiểu cho bus địa chỉ:

Ví dụ: Câu hỏi #bb55ef, IT3030---Midterm_with answers.pdf

Đề bài: Một hệ thống máy tính có bộ nhớ chính với dung lượng 2GB. Số bit tối thiểu cho bus địa chỉ là bao nhiêu?

Lời giải:

- Chuyển đổi dung lượng bộ nhớ sang byte: 2GB = $2 * 2^{30}$ byte = 2^{31} byte.
- Tính số bit địa chỉ: Để đánh địa chỉ 2^{31} byte, cần có 31 bit địa chỉ.

2. Xác định cách thức hoạt động của bộ nhớ cache

Xác định số bit cho trường Set và Tag trong bộ nhớ cache:

Ví dụ: Câu hỏi #c9ea55, IT3030---Midterm_with answers.pdf

Đề bài: Một bộ nhớ cache có 16KiB dữ liệu, sử dụng ánh xạ liên kết 2 chiều với kích thước khối 1 word. Bộ nhớ chính có 512MiB và sử dụng địa chỉ 32 bit. Xác định số bit của trường Tag và Set.

Lời giải:

- Kích thước khối: 1 word = 4 byte.
- Số khối trong cache: $16\text{KiB} / 4 \text{ byte/khối} = 4\text{K khối} = 2^{12}$ khối.
- Số set: Vì ánh xạ 2 chiều nên số set bằng số khối chia cho 2 = $2^{12} / 2 = 2^{11}$ set.
- Số bit cho trường Set: 11 bit.
- Số bit cho trường Word: 2 bit (vì mỗi khối có 4 byte = 2^2 byte).
- Số bit cho trường Tag: $32 - 11 - 2 = 19$ bit.

Xác định địa chỉ nhị phân, trường tag, trường index:

Ví dụ: Bài tập Exercise, IT3030E-CA-Chap6-Memory-Exercises.pdf

Đề bài: Cho biết địa chỉ 32 bit được chia thành các trường như sau:

```
| Tag | Index | Byte Offset |
|---|---|---|
```

và một cache có 2^{10} khối, mỗi khối 2^4 byte. Xác định số bit của mỗi trường.

Lời giải:

- Số bit cho trường Byte Offset: 4 bit (vì mỗi khối có 2^4 byte).

- Số bit cho trường Index: 10 bit (vì có 2^{10} khối).
- Số bit cho trường Tag: $32 - 10 - 4 = 18$ bit.

Xác định số bit cho direct-mapped cache:

Ví dụ: Bài tập 6.3, bai-tap-chuong-6.pdf

Đề bài: Cho máy tính với 64Kbytes bộ nhớ chính được đánh địa chỉ theo byte, bộ nhớ cache gồm 32 lines được tổ chức ánh xạ trực tiếp, kích thước mỗi line là 8 bytes.

a. Xác định số bit của các trường địa chỉ: Tag, Line, Word²

Lời giải:

- Số bit cho trường Word: 3 bit (vì mỗi line có 8 byte = 2^3 byte).
- Số bit cho trường Line: 5 bit (vì có 32 lines = 2^5 lines).
- Số bit cho trường Tag: $16 - 5 - 3 = 8$ bit (vì bộ nhớ chính có 64Kbytes = 2^{16} byte).

3. Tính toán và đánh giá hiệu năng bộ nhớ cache

Xác định tỷ lệ trùng cache:

Ví dụ: Bài tập Exercise, IT3030E-CA-Chap6-Memory-Exercises.pdf

Đề bài: Một chương trình có các lệnh được lưu trữ tuần tự trong bộ nhớ từ địa chỉ 0x1000 đến 0x101C.

Giả sử rằng mỗi lệnh chiếm 4 byte và bộ nhớ cache được tổ chức với 4 set, mỗi set 2 dòng, kích thước khối 4 byte. Ban đầu cache rỗng và chương trình được thực thi tuần tự. Xác định tỷ lệ trùng cache (hit ratio) khi chương trình được thực thi lần đầu tiên.

Lời giải:

1. Số khối trong cache: $4 \text{ set} * 2 \text{ dòng/set} = 8 \text{ khối}$.
2. Số byte trong cache: $8 \text{ khối} * 4 \text{ byte/khối} = 32 \text{ byte}$.
3. Số lệnh trong cache: $32 \text{ byte} / 4 \text{ byte/lệnh} = 8 \text{ lệnh}$.
4. Số lệnh trong chương trình: $(0x101C - 0x1000) / 4 + 1 = 8 \text{ lệnh}$.
5. Phân tích truy cập cache:
 - Lần đầu tiên thực thi chương trình, tất cả các lệnh đều bị miss cache.
 - Vì cache có thể chứa toàn bộ chương trình, nên từ lần thực thi thứ hai trở đi, tất cả các lệnh đều sẽ hit cache.
6. Tính tỷ lệ trúng cache:
 - Tỷ lệ miss cache lần đầu: $8 \text{ lệnh miss} / 8 \text{ lệnh} = 100\%$.
 - Tỷ lệ hit cache từ lần thứ hai: 100% .
 - Tỷ lệ hit cache trung bình: $(0\% + 100\%) / 2 = 50\%$.

Xác định tỷ lệ cache miss:

Ví dụ: Bài tập, Baitap2.pdf

Đề bài: Bộ nhớ chính 64 byte, bộ nhớ đệm 16 byte, kích thước line 4 byte, ánh xạ trực tiếp, cache ban đầu không chứa thông tin. Thứ tự truy cập bộ nhớ: 0 4 8 12 16 12 16 60.¹ Xác định tỷ lệ cache miss.

Lời giải:

1. Số dòng trong cache: $16 \text{ byte} / 4 \text{ byte/dòng} = 4 \text{ dòng}$.
2. Phân tích truy cập cache:
 - Truy cập 0: miss, nạp dòng chứa 0, 4, 8, 12 vào cache.
 - Truy cập 4: hit.
 - Truy cập 8: hit.
 - Truy cập 12: hit.
 - Truy cập 16: miss, nạp dòng chứa 16, 20, 24, 28 vào cache.
 - Truy cập 12: miss, nạp lại dòng chứa 0, 4, 8, 12 vào cache.
 - Truy cập 16: hit.

- Truy cập 60: miss, nạp dòng chứa 60, 64, 68, 72 vào cache.

3. Tính tỷ lệ cache miss: 6 lần miss / 8 lần truy cập = 75%.

So sánh hiệu năng của các thiết kế cache khác nhau:

Ví dụ: Bài tập Exercise, IT3030E-CA-Chap6-Memory-Exercises.pdf

Đề bài: So sánh hiệu năng của hai cache có cùng kích thước và cùng số khối, nhưng một cache sử dụng ánh xạ trực tiếp và cache còn lại sử dụng ánh xạ liên kết 2 chiều.

Lời giải:

- Ánh xạ trực tiếp: Mỗi khối trong bộ nhớ chính chỉ có thể được ánh xạ vào một dòng duy nhất trong cache. Ưu điểm là đơn giản, dễ dàng cài đặt. Nhược điểm là dễ xảy ra xung đột, dẫn đến tỷ lệ miss cache cao.
- Ánh xạ liên kết 2 chiều: Mỗi khối trong bộ nhớ chính có thể được ánh xạ vào một trong hai dòng trong một set. Ưu điểm là giảm xung đột, tăng tỷ lệ hit cache. Nhược điểm là phức tạp hơn ánh xạ trực tiếp.

Kết luận: Cache sử dụng ánh xạ liên kết 2 chiều thường có hiệu năng cao hơn cache sử dụng ánh xạ trực tiếp, đặc biệt là đối với các chương trình có tính chất truy cập bộ nhớ ngẫu nhiên.

4. Kiến trúc bộ nhớ

Kiến trúc băng nhớ đan xen:

Ví dụ: Câu hỏi #38a80d, IT3030---Midterm_with answers.pdf

Đề bài: Kiến trúc băng nhớ đan xen (interleaved memory) được sử dụng để làm gì?

Lời giải: Kiến trúc băng nhớ đan xen được sử dụng để tăng tốc độ truy cập bộ nhớ.

Trong kiến trúc này, bộ nhớ được chia thành nhiều băng nhớ (memory bank) hoạt

động độc lập. Các word liên tiếp nhau được lưu trữ ở các băng nhớ khác nhau, cho phép truy cập đồng thời nhiều word cùng lúc.

Xác định băng nhớ chứa địa chỉ:

Ví dụ: Câu hỏi #668120, Baitap2.pdf

Đề bài: Cho một máy tính có bus địa chỉ và bus dữ liệu có độ rộng 32-bit, lưu trữ theo kiểu đầu nhỏ. Máy tính sử dụng bộ nhớ RAM gồm 4 băng nhớ thiết kế theo kiểu băng nhớ đan xen. Hãy cho biết² ngăn nhớ ở địa chỉ 0x1F562D6E nằm trên băng nhớ nào?

Lời giải:

- Lấy hai bit thấp nhất của địa chỉ: 0x1F562D6E = 1111101011001101111001101110₂. Hai bit thấp nhất là 10₂ = 2.
- Kết luận: Ngăn nhớ ở địa chỉ 0x1F562D6E nằm trên băng nhớ số 2.